Team -8

Stage – 2-1: Data Preparation

Raman Shrestha
Rahul Sarker
Jyoti Saini
Bao Ngoc Nguyen
Fahad Pasha Mohammed

CONTENTS

1. Introduction and Objectives: 2
2. Data Preparation and Preprocessing 3
2.1 Tools and Libraries 3
2.2 Loading the Dataset 3
2.3 Initial Data Exploration 3
2.4 Data Quality Assessment 5
2.5 Removing Duplicate Records 6
2.6 Categorical Data Inspection and Standardization 6
2.7 Converting selected fields to numeric data types 7
2.8 Numerical Validation 8
2.9 Understanding the churn target variable 8
2.10 Potential Outlier Detection Using IQR Method 9
2.11 Correlation Analysis of Numerical Variables 10
2.12 Encoding Categorical variables with One-Hot Encoding 11
2.13 Standardising Numerical Variables 12
2.14 Converting Churn into binary numerical values 12
2.15 Separating Predictors and Target 13
2.16 Splitting the data into Training and Testing Sets 13
2.17 Scaling the Training and test datasets 14
2.18 Exporting Training and Testing into csv: 14
2.19 Creating a training and testing summary 15
2.20 Final Data Quality Verification 16
2.21 Exporting the Processed Dataset 16
Summary of the Preprocessing Workflow 18
Conclusion 19
Appendix A – Notebook Output 20

1. Introduction and Objectives:

Customer churn is a constant problem in the modern telecommunications industry, affecting revenue stability, market share and longer-term growth. This report is designed to set up a robust data preprocessing pipeline along with the use of unsupervised K-Means customer segmentation to analyse customer behavioural, contractual and financial profiles. The main goals is to maintain absolute data integrity, divide the subscribers into four segments (K=4), and discover high-risk segments to serve as the basis for targeted business retention actions and operational decisions.

Telecommunications company constantly face customer attrition: When customers cancel their service, they're lost for good. And it is easier and less costly to keep a customer than to acquire one. The customers that were analysed in this dataset, 1869 (26.5%) have churned. This report does not show differences between individual customers and uses K-Means clustering to segment customers and identify where the churn is occurring.

The given dataset contains 7043 instances with 10 features such as gender, Senior Citizen, Dependents, tenure, phone services, Multiple Lines, Internet Service, Contract, Monthly Charges and churn as the target variable. It lacks any customer ID, billing history or usage information. The goal is to generate clean data for clustering and a subsequent ANN model; to determine and explain the number of segments and their description in business terms; and to determine which segments are losing the most customers for which initial retention opportunities can be suggested.

2. Data Preparation and Preprocessing

2.1 Tools and Libraries

Library / function	Role in the workflow	Business analytics relevance
Pandas (pd)	Loads, inspects, filters, transforms and exports tabular data.	Provides the main tools for managing customer records and attributes.
NumPy (np)	Supports numerical operations and selection of numerical data types.	Helps identify numerical fields for statistical analysis.
Matplotlib (plt)	Creates the correlation heatmap.	Make relationships among numerical variables easier to inspect.
StandardScaler	Standardize selected numerical variables.	Places variables on comparable scales for algorithms sensitive to feature magnitude.
train_test_split	Divides data into training and testing subsets.	Supports evaluation of a predictive model on held-out records.

2.2 Loading the Dataset

Importing the CSV file:

```
df = pd.read_csv("Dataset_ATS_v2.csv")
```

The pd.read_csv() function reads the comma-separated values file and loads it into a Pandas DataFrame named df. A DataFrame stores the data in rows and columns. In this project, each row represents a customer record, and each column represents a customer attribute.

Loading the data into a DataFrame makes it possible to inspect, clean, transform and analyse the information using Python. The file path must point to the correct location of the CSV file in the Jupyter environment.

Business interpretation: This step establishes the working dataset from which customer characteristics and churn outcomes can be examined.

2.3 Initial Data Exploration

The head() function displays the first five rows by default. It provides a quick visual check that the file has loaded and allows the analyst to see how the values are organised across columns.

The preview can reveal obvious formatting issues, unexpected values or column contents that require further investigation. It is an initial inspection rather than a complete quality assessment.

```
df.head()

df.shape
```

The shape attribute returns a tuple containing the number of rows and columns. The first value is the number of records and the second is the number of variables. For example, an output of (7043, 10) would mean 7,043 records and 10 columns; this is an illustration and must be replaced with the actual notebook output.

The dimensions establish the baseline size of the raw dataset. Comparing this value with later row counts can help identify whether records were removed during subsequent processing.

Reviewing sample customer records helps the analyst understand what customer information is available for studying churn.

```
df.columns

df.info()

df.describe()
```

This displays the names of all variables in the DataFrame. It helps confirm which customer attributes are available and ensures that the correct column names are referenced in later code.

Examples in the supplied workflow include gender, SeniorCitizen, Dependents, tenure, PhoneService, MultipleLines, InternetService, Contract, MonthlyCharges and Churn. The actual list should be verified against the notebook output.

The info() method provides a structural summary of the DataFrame. It reports the column names, non-null counts, data types and memory usage. The non-null count is useful for spotting columns that may contain missing observations.

Data type	Meaning	Example in this type of dataset
int64	Whole-number numerical values	SeniorCitizen, if stored as 0 or 1
float64	Numerical values that can include decimals	MonthlyCharges
object / string	Text or categorical values	gender, Contract, Churn
bool	True/False values	Possible encoded indicator columns

The data type matters because numerical calculations and machine learning algorithms expect appropriate input formats. A number stored as text may not be available for mathematical operations until it is converted.

For numerical columns, describe() returns count, mean, standard deviation, minimum, 25th percentile, median (50th percentile), 75th percentile and maximum.

Statistic	Meaning	How it helps this project
count	Number of non-missing observations	Shows how many numerical values are available.
mean	Arithmetic average	Summarizes typical values, such as average monthly charges.
std	Standard deviation	Shows the spread or variability of values.
min / max	Smallest and largest values	Can reveal implausible or unusual values.
25% / 50% / 75%	Quartiles and median	Describe the distribution and support IQR outlier checks.

The statistics help the analyst understand the scale and distribution of fields such as tenure and MonthlyCharges. The mean can be influenced by extreme values, so it should be interpreted alongside the median and quartiles.

Descriptive statistics provide an initial profile of customers, including how long they have stayed and the range of monthly charges.

2.4 Data Quality Assessment

```
df.isnull().sum()

df.duplicated().sum()
```

isnull() identifies values recognized by Pandas as missing, and sum() counts those missing values in each column. This allows the analyst to identify fields that may be incomplete.

Missing values can affect summaries, visualisations and modelling. Depending on the field and the reason for missingness, possible treatments include removing affected records, imputing numerical values using a suitable statistic such as the median, or imputing categorical values using the mode or an explicit unknown category.

duplicated() identifies rows that are exact duplicates of an earlier row, and sum() counts them. Duplicate records may cause some observations to receive unintended extra weight in analysis.

Duplicates should be investigated before removal. Two rows that look alike may represent separate customers if the dataset has no unique customer identifier, while exact accidental duplicates may need to be removed. If confirmed as accidental duplicates, Pandas can remove them using df = df.drop_duplicates().

2.5 Removing Duplicate Records

```
before = len(df)

df = df.drop_duplicates()

removed = before - len(df)

print(f"Duplicate rows removed: {removed}")
```

It stores the original number of rows before duplicate removal. It also retains the first occurrence by default and removes repeated, identical rows. The cleaned dataset is again assigned back to df. removed= before – len (df) calculates how many records were removed.

The step reported that there were 302 duplicate records. The expected remaining number would be:

$$7043 - 302 = 6741$$

Removing confirmed duplicate customer observations helps prevent repeated records from receiving additional weight during analysis and predictive modelling. However, this assumes that exact duplicate rows represent accidental duplication rather than legitimate repeated observations.

2.6 Categorical Data Inspection and Standardization

Inspecting Contract categories

unique() returns the distinct values found in the Contract column. It helps reveal the categories represented in the data and can identify unexpected labels or formatting variations.

Contract categories commonly distinguish month-to-month agreements from one-year and two-year agreements. The actual categories must be taken from the notebook output.

```
df["Contract"].unique()
```

Contract duration may be relevant to customer commitment and retention. Examining the categories confirms that contract types are represented consistently before analysis.

Trimming whitespace from text fields

select_dtypes(include=['object','string']) selects columns stored as text. The for loop iterates over those columns, and .str.strip() removes whitespace from the beginning and end of each text value.

For example, " Yes " becomes "Yes". Without trimming, values that look identical to a person may be treated as different categories by Python, which can create extra categories during analysis or encoding.

```
for col in df.select_dtypes(include=["object","string"]).columns:
    df[col] = df[col].str.strip()
```

This operation does not remove spaces within a phrase, such as the space in 'Month-to-month'; it removes only leading and trailing whitespace.

Standardizing text formatting helps ensure that customers with the same response are grouped consistently.

Verifying selected categorical values

```
columns_to_check = ["gender", "Dependents", "PhoneService", "MultipleLines", "InternetService", "Churn"]

for col in columns_to_check:
    print(f"--- Unique values in {col} ---")
    print(df[col].unique())
    print()
```

The list identifies the categorical columns selected for inspection. The loop prints each column name and its distinct values, with blank lines to make the output easier to read.

This is a validation step: it checks whether the categories are expected and whether inconsistent labels remain after whitespace trimming.

For example, MultipleLines may include 'No phone service'. This should not automatically be treated as equivalent to 'No', because it can represent a customer who does not have phone service at all.

Correct category definitions matter when comparing customer groups. Preserving meaningful distinctions avoids combining customers whose service circumstances differ.

2.7 Converting selected fields to numeric data types

```
df["SeniorCitizen"] = pd.to_numeric(
    df["SeniorCitizen"],
    errors = 'coerce'
)

df["tenure"] = pd.to_numeric(
    df["tenure"],
    errors = 'coerce'
)

df["MonthlyCharges"] = pd.to_numeric(
    df["MonthlyCharges"],
    errors = 'coerce'
)
```

pd.to_numeric() attempts to convert the selected columns into numerical values. The errors='coerce' parameter converts any value that cannot be interpreted as a number into NaN.

Variable	Business meaning	Why numeric format is required
SeniorCitizen	Indicator of whether the customer is a senior citizen (often 0 or 1).	Allows numerical operations and inclusion in numerical transformations.

tenure	Length of the customer's relationship with the business, generally measured in months.	Supports summaries, relationship analysis and scaling.
MonthlyCharges	Recurring monthly amount charged to the customer.	Supports financial summaries, outlier detection and modelling.

This conversion is necessary for calculations such as mean, minimum, maximum, correlation and standardization. However, coercing invalid values to NaN can create missing values, so the columns should be checked for missing values again after conversion. Correct numerical formats allow the analyst to assess customer tenure and billing patterns quantitatively.

2.8 Numerical Validation

Describing Monthly Charges

This generates descriptive statistics specifically for MonthlyCharges. It provides the count, mean, standard deviation, minimum, quartiles and maximum for customer monthly charges.

```
df["MonthlyCharges"].describe()
```

The output helps the analyst understand typical customer billing, variability in charges and whether unusually high or low values may require investigation.

Monthly charges may reflect the services customers have selected. Understanding their distribution can support later investigation of whether billing levels are associated with churn.

Checking for negative monthly charges, negative tenure and maximum tenure:

```
(df["MonthlyCharges"] < 0).sum()

(df["tenure"] <0).sum()

df["tenure"].max()
```

The expression tests each MonthlyCharges value to see whether it is below zero. The comparison produces True or False values, and sum() counts the True results.

This counts records where tenure is negative. The check helps protect retention analysis from invalid relationship-duration values. The code only counts such records and does not correct them.

max() returns the largest tenure value in the dataset. This helps establish the longest recorded customer relationship and assess whether the range is plausible for the business context.

2.9 Understanding the Churn Target Variable

```
df["Churn"].value_counts()
```

value_counts() counts the number of records in each category of Churn. In a typical binary churn dataset, the categories indicate whether a customer has churned (Yes) or not churned (No).

This gives the analyst the number of customers in each outcome group and provides a first view of the balance between retained and churned customers.

```
df["Churn"].value_counts(normalize=True) * 100
```

normalize=True converts category counts into proportions of the total, and multiplying by 100 expresses those proportions as percentages. This makes the distribution easier to communicate and compare.

A dataset in which one outcome is much more common than the other may have class imbalance. In churn modelling, a model that predicts the majority class too often may appear accurate while failing to identify customers who actually churn. The churn percentage provides an initial measure of the scale of customer loss in the dataset. It can help frame the business problem and inform the evaluation metrics used later.

2.10 Potential Outlier Detection Using IQR Method

```
Q1 = df["MonthlyCharges"].quantile(0.25)
Q3 = df["MonthlyCharges"].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = df[
    (df["MonthlyCharges"] < lower) |
    (df["MonthlyCharges"] > upper)
]

print(outliers.shape)
```

The Interquartile Range method identifies potential outliers by examining how far values fall from the middle 50% of a distribution. The method is applied here to MonthlyCharges.

Q1 is the 25th percentile. Approximately one quarter of the observations are at or below this value.

Q3 is the 75th percentile. Approximately three quarters of the observations are at or below this value.

The IQR measures the spread of the middle 50% of monthly charges. It is calculated by subtracting Q1 from Q3.

The conventional IQR rule defines values below Q1 − 1.5 × IQR or above Q3 + 1.5 × IQR as potential outliers.

The filter selects rows where MonthlyCharges is below the lower boundary OR above the upper boundary. The vertical bar (|) represents the logical OR operator.

The shape of the filtered DataFrame reports the number of flagged records and the number of columns in those records.

Higher monthly charges may be legitimate because customers can subscribe to different combinations or levels of service. Removing these records automatically could discard valuable customer behaviour. The appropriate action is to investigate flagged records and retain valid observations.

2.11 Correlation Analysis of Numerical Variables

```
numeric_df = df.select_dtypes(include=np.number)

plt.figure(figsize=(10, 8))
plt.imshow(numeric_df.corr(), cmap="coolwarm")
plt.colorbar()
plt.xticks(range(len(numeric_df.columns)),
           numeric_df.columns,
           rotation=90)
plt.yticks(range(len(numeric_df.columns)),
           numeric_df.columns)
plt.show()
```

This section selects numerical columns, calculates pairwise correlations and displays the resulting matrix as a heatmap. Correlation measures the strength and direction of a linear relationship between two numerical variables.

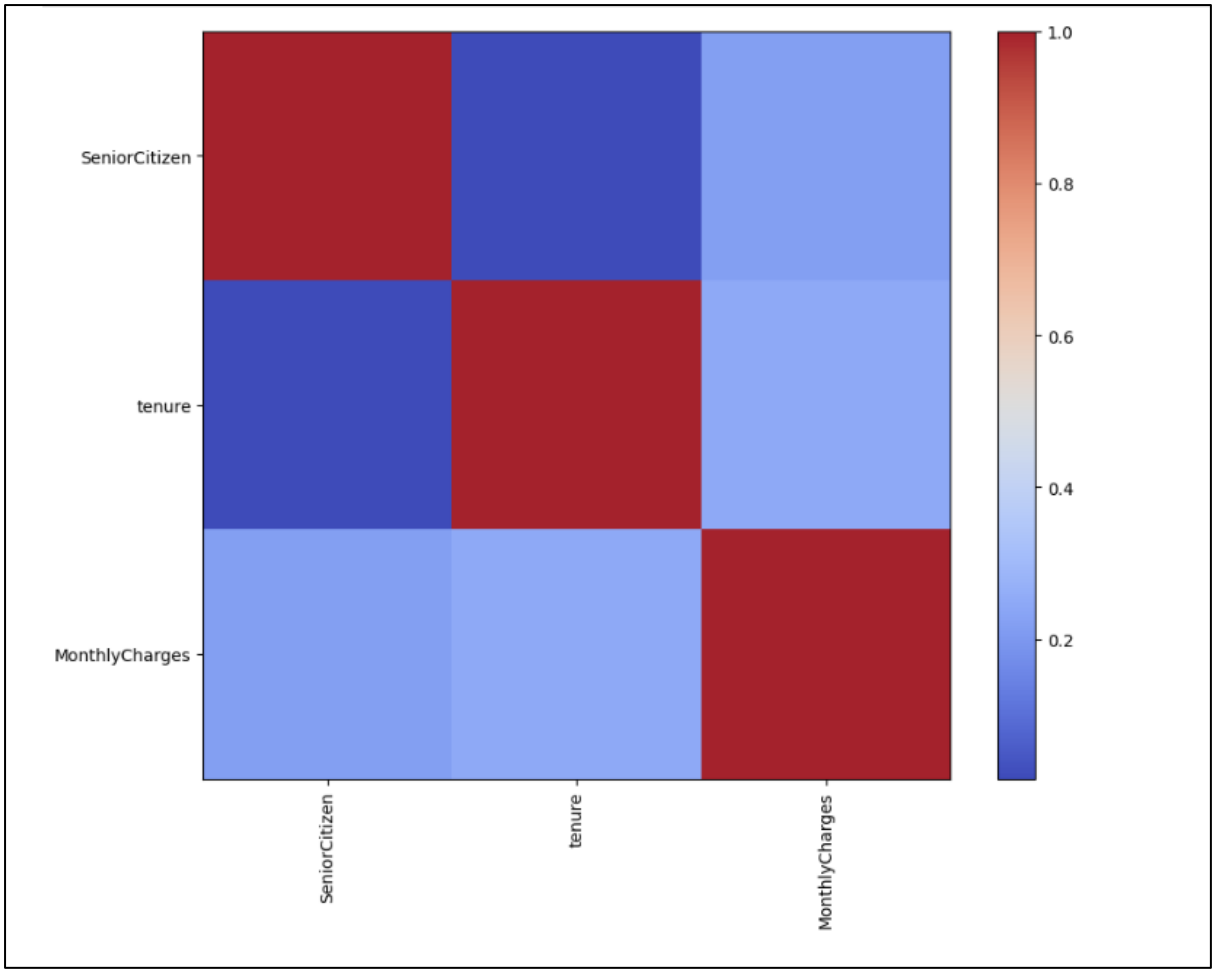
select_dtypes(include=np.number) retains columns recognised as numerical and excludes text-based categorical columns. This ensures that the correlation calculation is applied to numerical data.

This creates a Matplotlib figure that is 10 inches wide and 8 inches high. The size provides room for the variable labels and the heatmap.

corr() calculates the correlation matrix. imshow() displays the matrix as a colour-coded image. The coolwarm colour map helps distinguish positive and negative relationships.

Correlation coefficient	General interpretation
Close to +1	Strong positive linear association: higher values of one variable tend to occur with higher values of the other.
Around 0	Little or no linear association.
Close to -1	Strong negative linear association: higher values of one variable tend to occur with lower values of the other.

colorbar() adds a scale that maps colours to correlation values. xticks() labels the horizontal axis and rotates labels by 90 degrees to improve readability. yticks() labels the vertical axis. show() renders the completed visualisation.



The amount of correlation is modest: the correlation between tenure and churn is -0.35; the correlation between monthly charges and churn is +0.19; there are no correlations above 0.25 between any pair of input variables, so there is no multicollinearity. Each variable may provide different information for customer churn prediction. However, further analysis is required to identify which factors are most strongly associated with customer churn.

2.12 Encoding Categorical variables with One-Hot Encoding

Many machine learning algorithms require numerical input. One-Hot Encoding converts a categorical variable into a set of binary indicator columns. Each indicator represents a category and takes a value of 1 when the record belongs to that category and 0 otherwise.

The columns parameter identifies which categorical variables are to be encoded. In this workflow, PhoneService, MultipleLines, InternetService and Contract are selected.

```
df = pd.get_dummies(
    df,
    columns = ["PhoneService", "MultipleLines", "InternetService", "Contract"],
    drop_first = True
)
```

drop_first=True drops the first category from each encoded variable. For a variable with k categories, this generally leaves k−1 indicator columns. The omitted category becomes the reference category,

represented when all retained indicators for that variable are zero. This can reduce redundant information in some modelling settings. Encoding allows a predictive model to use information about service subscriptions and contract types. It can help the model learn whether churn outcomes differ across customer service categories.

2.13 Standardising Numerical Variables

StandardScaler transforms selected numerical variables so that each has a mean close to zero and a standard deviation close to one, based on the data used to fit the scaler.

```
scaler = StandardScaler()

numerical_columns = [
    "SeniorCitizen",
    "tenure",
    "MonthlyCharges"
]

df[numerical_columns] = scaler.fit_transform(
    df[numerical_columns]
)
```

This creates a StandardScaler object from scikit-learn. The object will calculate the mean and standard deviation for each selected feature when fit_transform() is called. This list specifies the variables to be standardised. SeniorCitizen is commonly a binary indicator, tenure is a duration measure and MonthlyCharges is a monetary measure.

fit_transform() performs two operations: fit calculates the mean and standard deviation from the supplied data, and transform applies the standardisation formula to each value.

The formula is $Z = (X - \mu) / \sigma$, where X is the original value, μ is the feature mean and σ is the feature standard deviation. The resulting values are expressed in standard deviation units relative to the mean.

Standardisation is often useful for models such as Artificial Neural Networks because features measured on very different scales can make optimisation more difficult. Standardisation places selected numerical attributes on comparable scales, helping prepare them for modelling. The transformed values are mathematical representations; they no longer retain the original units such as months or dollars.

2.14 Converting Churn into Binary Numerical Values

```
df["Churn"] = df["Churn"].map({
    "No": 0,
    "Yes": 1
})
```

The original churn column contains text categories Yes or No. The .map() function converts these categories into numerical values.

Original Value	Encoded Value
No	0
Yes	1

This process is called binary encoding.

The reason for doing this is that a machine learning model such as an Artificial Neural Network requires the target variable to be represented numerically for a standard binary classification setup.

Converting churn into 0 and 1 allows the predictive model to learn patterns associated with whether a customer is likely to leave the service provider.

The encoding also makes the results easier to interpret:

- 0 represents a retained customer.
- 1 represents a churned customer.

2.15 Separating Predictors and Target

```
x = df.drop("Churn", axis = 1)
y = df["Churn"]
```

This step separates the dataset into independent variables (X) and the dependent or target variable (y). drop('Churn', axis=1) removes the Churn column from the DataFrame. axis=1 specifies that a column, rather than a row, is being dropped. The remaining customer attributes form the predictor dataset.

Dataset component	Contents	Purpose
X – predictors	Customer characteristics such as tenure, charges, demographic indicators and service/contract features.	Inputs supplied to the model.
y – target	Customer churn outcome.	Outcome the model learns to predict.

2.16 Splitting the Data into Training and Testing Sets

```
x_train, x_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size = 0.2,
    random_state = 42,
    stratify = y
)
```

The train_test_split() function divides the predictors and target into corresponding training and testing subsets. The training set is used to fit the model; the testing set is held back to evaluate performance on records not used for fitting.

test_size=0.2 assigns 20% of the observations to the test set and the remaining 80% to the training set. The exact record counts depend on the number of usable rows at the time of splitting. For 7,043 rows, the approximate split would be 5,634 training records and 1,409 test records, subject to rounding.

random_state=42 fixes the random seed used for the split. This makes the split reproducible when the same data and settings are used, which is useful for consistent analysis and reporting. The number 42 is a chosen seed; it has no special statistical meaning.

stratify=y instructs the function to preserve approximately the same class proportions in the training and testing subsets as in the full target variable. This is useful when churn and non-churn customers are not

equally represented. The training data allows the model to learn patterns from historical customers, while the test data provides an independent check of how well those patterns generalise to unseen customer records.

2.17 Scaling the Training and test datasets

```
x_train.loc[:, numerical_columns] = scaler.fit_transform(
    x_train[numerical_columns]
)

x_test.loc[:, numerical_columns] = scaler.transform(
    x_test[numerical_columns]
)
```

This applies standardisation to the numerical columns in the training dataset.

The fit_transform() method:

1. Calculates the mean and standard deviation using the training data.
2. Uses these values to standardise the training features.
3. Stores the transformed values back into x_train.

The .loc[:, numerical_columns] expression selects all rows and only the specified numerical columns. The important difference is that this command uses transform() rather than fit_transform().

It does not calculate new means or standard deviations from the testing data.

Instead, it uses the scaling parameters already learned from the training dataset.

The testing dataset must remain independent of the training process. If the test data is used to calculate scaling parameters, information from the test set can influence model preparation.

Applying scaling based only on training data helps ensure that model evaluation is more realistic and reduces the risk of data leakage.

2.18 Exporting Training and Testing into csv:

```
x_train.to_csv("X_train.csv", index = False)
x_test.to_csv("X_test.csv", index = False)
y_train.to_frame(name = "Churn").to_csv("Y_train.csv", index = False)
y_test.to_frame(name = "Churn").to_csv("Y_test.csv", index = False)
```

After the dataset is split and scaled, the following code saves the four datasets into separate CSV files.

This saves the independent variables used for model training into a file called X_train.csv, This saves the independent variables reserved for testing into X_test.csv,

The to_frame() method converts the target Series into a DataFrame. The name="Churn" parameter assigns the column name Churn. The resulting DataFrame is then exported as Y_train.csv and Y_test.csv.

Understanding the four output files

File	Contents	Purpose
X_train.csv	Training customer features	Input for model training
X_test.csv	Testing customer features	Input for model evaluation
Y_train.csv	Training churn labels	Correct outcomes for training
Y_test.csv	Testing churn labels	Actual outcomes for testing predictions

The parameter index=False prevents Pandas from exporting the DataFrame index as an additional column.

Saving the datasets separately makes the preprocessing workflow reusable. It also provides a clear separation between the information used to train the model and the information reserved to assess its performance.

2.19 Creating a training and testing summary

```
split_summary = pd.DataFrame({
    "Dataset" : ["Training", "Testing"],
    "Rows" : [len(x_train), len(x_test)],
    "Features" : [x_train.shape[1], x_test.shape[1]],
    "Churn_Yes" : [
        int((y_train == 1).sum()),
        int((y_test == 1).sum())
    ],
    "Churn_No" : [
        int((y_train == 0).sum()),
        int((y_test == 0).sum())
    ]
})

split_summary.to_csv("Tran_Test_Summary.csv", index = False)
```

This section creates a summary table to compare the training and testing datasets. The results are stored in a new Pandas DataFrame called split_summary

The following code identifies whether the row represents the training or testing dataset then calculate number of records in each dataset.

Churn_Yes:

```
int((y_train == 1).sum())
int((y_test == 1).sum())
```

These expressions count the customers who churned in each dataset. The comparison y_train == 1 identifies churned customers, and .sum() counts them.

Churn_No:

```
int((y_train == 0).sum())
int((y_test == 0).sum())
```

These expressions count the customers who did not churn in each dataset. The int() function converts the count into an integer.

```
split_summary.to_csv("Tran_Test_Summary.csv", index=False)
```

This exports the summary table into CSV file called Tran_Test_Summary.csv.

This summary provides an overview of how customer records have been distributed between the training and testing datasets. It also allows the analyst to confirm that both datasets contain churned and non-churned customers. It is particularly useful for checking whether stratification has maintained a similar churn distribution across both groups.

2.20 Final Data Quality Verification

```
print("Final quality check:")

print("\nMissing values:")
print(df.isnull().sum())

print("\nDuplicates:")
print(df.duplicated().sum())

print("\nData types:")
print(df.dtypes)

print("\nChurn distribution:")
print(df["Churn"].value_counts())
```

The final quality check reviews key properties of the transformed DataFrame before export. The newline characters (\n) create spacing in the printed output to make each check easier to read. df.isnull().sum() counts missing values in each column after the type conversions and transformations. This is especially important because errors='coerce' may have converted invalid numeric entries into NaN.

df.duplicated().sum() counts exact duplicate rows in the processed dataset. The code reports the count but does not remove duplicates. df.dtypes displays the type of each column. This helps confirm that numerical transformations and dummy-variable encoding have produced the expected types. It also helps reveal any text columns that remain. df['Churn'].value_counts() confirms the number of records in each target category after preprocessing. This helps check that the target remains present and that its distribution has not unexpectedly changed. These checks provide a final review of the dataset's state.

2.21 Exporting the Processed Dataset

to_csv() writes the current DataFrame to a CSV file named Updated_Dataset.csv. The index=False parameter prevents Pandas from writing the DataFrame row index as an additional column.

```
df.to_csv('Updated_Dataset.csv', index = False)
```

Exporting the file creates a reusable output from the preprocessing workflow. It can be loaded into later notebooks or analytical processes without repeating all earlier steps, provided the preprocessing decisions are documented and reproducible.

SUMMARY OF THE PREPROCESSING WORKFLOW

Stage	Operation performed	Outcome / purpose
1. Import	Load Python libraries and read the CSV.	Create the working DataFrame.
2. Explore	head, shape, columns, info and describe.	Understand records, variables, data types and distributions.
3. Assess quality	Check null values and duplicate rows.	Identify potential completeness and duplication issues.
4. Duplicate check and removal	Used duplicated().sum() and drop_duplicates()	Identified and removed exact duplicate records to reduce repeated observations and improve data quality.
5. Inspect categories	Check Contract and selected unique values.	Validate category labels and expected values.
6. Standardise text	Strip leading/trailing whitespace.	Reduce inconsistent category labels.
7. Convert types	Convert SeniorCitizen, tenure and MonthlyCharges to numeric.	Enable numerical operations.
8. Validate numbers	Check charges and tenure for negative values; inspect maximum tenure.	Identify potentially invalid numerical values.
9. Examine target	Count churn categories and calculate percentages.	Understand the target distribution and potential class imbalance.
10. Detect outliers	Apply IQR to MonthlyCharges.	Flag unusual values for investigation.
11. Correlation	Create numerical correlation heatmap.	Explore linear relationships among numerical variables.
12. Encode categories	One-hot encode four selected categorical fields.	Convert selected text categories to numeric indicators.
13.Target Variable Encoding	Converted Churn values from No/Yes into 0/1	Prepared the target variable for binary classification, where 0 represents customers who stayed and 1 represents customers who churned.
14. Scale features	Apply StandardScaler to three numerical fields.	Place selected variables on a comparable scale.
15. Define modelling data	Separate X and y.	Identify predictors and target.
16. Split data	80:20 split with random_state=42 and stratify=y.	Prepare training and testing subsets.
17. Dataset Export	Exported training, testing, target and summary CSV files	Saved the processed datasets for subsequent modelling and evaluation.
18. Final checks	Recheck missing values, duplicates, data types and churn counts.	Review the transformed dataset.
19. Export	Save Updated_Dataset.csv.	Create a reusable processed data file.

CONCLUSION

The data preparation and cleaning workflow establishes the foundation for the customer churn analysis project. It begins by loading and exploring the raw dataset, then reviews its dimensions, variables, data types and descriptive statistics. Missing values and duplicate records were checked, and duplicate rows were removed to improve the reliability of the dataset.

Categorical values were inspected and cleaned by removing unnecessary spaces. Numerical columns, including SeniorCitizen, tenure and MonthlyCharges, were converted into appropriate numeric formats. Additional validation was performed to identify negative values and examine the distribution of monthly charges. The IQR method was used to detect potential outliers in MonthlyCharges, while a correlation heatmap helped identify relationships between numerical variables.

To make the dataset suitable for machine learning, selected categorical variables were converted into dummy variables, and the Churn column was encoded into binary values, with 0 representing customers who stayed and 1 representing customers who left. The independent and dependent variables were then separated, followed by an 80:20 training and testing split using stratification to maintain a representative churn distribution.

Finally, the processed datasets and training/testing summary were exported into CSV files for use in the next stages of the project.

This preprocessing stage establishes a more consistent and structured dataset for identifying customer behaviour and understanding the factors associated with customer churn. The prepared data will support the next stage of the project, which involves developing a customer churn analysis and evaluating predictive models, including an Artificial Neural Network (ANN). The objective is to identify customers who are more likely to discontinue their services and provide meaningful insights that can help the business develop effective customer retention strategies.

APPENDIX A – NOTEBOOK OUTPUT

```
<class 'pandas.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 10 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   gender           7043 non-null   str
 1   SeniorCitizen    7043 non-null   int64
 2   Dependents       7043 non-null   str
 3   tenure           7043 non-null   int64
 4   PhoneService     7043 non-null   str
 5   MultipleLines    7043 non-null   str
 6   InternetService  7043 non-null   str
 7   Contract         7043 non-null   str
 8   MonthlyCharges   7043 non-null   int64
 9   Churn            7043 non-null   str
dtypes: int64(3), str(7)
memory usage: 357.7 KB
Duplicate rows removed: 302
--- Unique values in gender ---
<StringArray>
['Female', 'Male']
Length: 2, dtype: str

--- Unique values in Dependents ---
<StringArray>
['No', 'Yes']
Length: 2, dtype: str

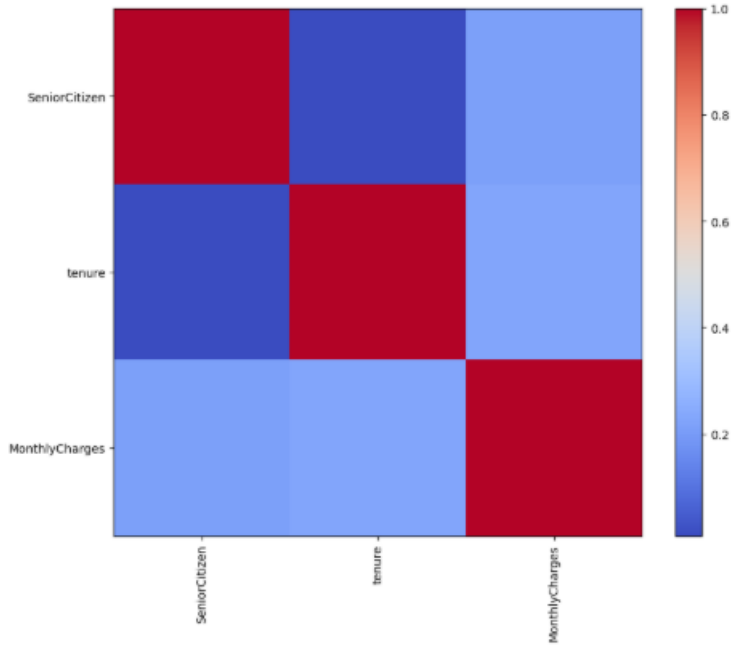
--- Unique values in PhoneService ---
<StringArray>
['No', 'Yes']
Length: 2, dtype: str

--- Unique values in MultipleLines ---
<StringArray>
['No', 'Yes']
Length: 2, dtype: str

--- Unique values in InternetService ---
<StringArray>
['DSL', 'Fiber optic']
Length: 2, dtype: str

--- Unique values in Churn ---
<StringArray>
['Yes', 'No']
Length: 2, dtype: str

(0, 10)
```




```
Final quality check:

Missing values:
gender                          0
SeniorCitizen                   0
Dependents                      0
tenure                          0
MonthlyCharges                  0
Churn                           0
PhoneService_Yes                0
MultipleLines_Yes               0
InternetService_Fiber optic     0
Contract_One year               0
Contract_Two year               0
dtype: int64

Duplicates:
0

Data types:
gender                           str
SeniorCitizen                float64
Dependents                       str
tenure                       float64
MonthlyCharges               float64
Churn                          int64
PhoneService_Yes                bool
MultipleLines_Yes               bool
InternetService_Fiber optic     bool
Contract_One year               bool
Contract_Two year               bool
dtype: object
```

```
Churn distribution:
Churn
0    4950
1    1791
Name: count, dtype: int64
```